perplexity

# # □ Medium (Applied Concepts) – Java OOPS

### 11. Explain constructor chaining in inheritance. How does `super()` work?

- Constructor chaining is the process where one constructor calls another (either in the same class or in the parent class).
- `this()` → calls another constructor in the same class.
- `super()` → calls the constructor of the parent class.

```
class A {
    A() {
        System.out.println("Parent constructor");
    }
}

class B extends A {
    B() {
        super(); // calls parent constructor
        System.out.println("Child constructor");
    }
}

public class Test {
    public static void main(String[] args) {
        new B();
    }
}
```

✕ No.

Abstract → meant to be extended by subclasses.

Final → prevents inheritance.

Both together contradict each other.

13. What happens if a subclass defines a field with the same name as the parent?

Fields in Java are not overridden but shadowed.

Which field is accessed depends on the reference type, not the object type.

class Parent {
int x = 10;

```
}

class Child extends Parent {
int x = 20;
}

public class Test {
public static void main(String[] args) {
Parent p = new Child();
System.out.println(p.x); // prints 10 (parent's field)
}
}
```

14. Explain access modifiers in inheritance (private, protected, default, public`).

private → not accessible outside the class (not inherited).

default (package-private) → accessible only within the same package.

protected → accessible within the same package + by subclasses.

public → accessible from anywhere.

15. Can we achieve multiple inheritance in Java? If yes, how?

✖ Not with classes → Java avoids diamond problem.

✅ Possible with interfaces.

```
interface A { void methodA(); }
interface B { void methodB(); }

class C implements A, B {
public void methodA() { System.out.println("A"); }
public void methodB() { System.out.println("B"); }
}
```

16. Explain the concept of covariant return types in overriding.

When overriding, the return type can be more specific than the parent method's return type.

```
class Animal {}
class Dog extends Animal {}

class Parent {
Animal getAnimal() { return new Animal(); }
}

class Child extends Parent {
@Override
Dog getAnimal() { return new Dog(); } // covariant return
}
```

17. What is the difference between composition and inheritance? Which one is preferred?

Inheritance → “is-a” relationship (Dog is an Animal).

Composition → “has-a” relationship (Car has an Engine).

✅ Composition is preferred (looser coupling, more flexibility).

18. How does instanceof operator work with inheritance?

Checks at runtime whether an object is an instance of a class or its subclass.

Animal a = new Dog();
System.out.println(a instanceof Animal); // true
System.out.println(a instanceof Dog); // true

19. Explain the Template Method Pattern with an example in OOPS.

Defines the skeleton of an algorithm in a parent class but lets subclasses override specific steps.

abstract class Payment {
public final void makePayment() {
validate();
process();
receipt();
}

```
  abstract void process();

  void validate() { System.out.println("Validating payment"); }
  void receipt() { System.out.println("Printing receipt"); }
```

}

class CardPayment extends Payment {
void process() { System.out.println("Processing card payment"); }
}

20. What are default methods in interfaces? Why were they introduced?

Introduced in Java 8.

Allow interfaces to have method implementations.

Purpose:

Backward compatibility → old interfaces can add new methods without breaking existing classes.

Enable multiple inheritance of behavior.

interface Vehicle {
default void start() {

System.out.println("Vehicle starting...");
}
}

convert above to .md format

Got it ✅
Here's your full content converted into a clean **Markdown (.md)** format, with proper headings, code formatting, and explanations for **Q11–Q20**.

# 🟡 Medium (Applied Concepts) – Java OOPS

### 11. Explain constructor chaining in inheritance. How does `super()` work?

- **Constructor chaining** is the process where one constructor calls another (either in the same class or in the parent class).
- `this()` → calls another constructor in the same class.
- `super()` → calls the constructor of the parent class.

```
class A {
    A() {
        System.out.println("Parent constructor");
    }
}

class B extends A {
    B() {
        super(); // calls parent constructor
        System.out.println("Child constructor");
    }
}

public class Test {
    public static void main(String[] args) {
        new B();
    }
}
```

### 12. Can a class be both abstract and final? Why/Why not?

❌ No.

- **Abstract** → meant to be extended by subclasses.
- **Final** → prevents inheritance.
- Both contradict each other, so a class cannot be both.

### 13. What happens if a subclass defines a field with the same name as the parent?

- Fields in Java are **not overridden** (only methods can be) → they are **shadowed**.
- Which field is accessed depends on the **reference type**, not the object type.

```
class Parent {
    int x = 10;
}

class Child extends Parent {
    int x = 20;
}

public class Test {
    public static void main(String[] args) {
        Parent p = new Child();
        System.out.println(p.x); // prints 10 (parent's field)
    }
}
```

### 14. Explain access modifiers in inheritance (`private`, `protected`, `default`, `public`).

- **private** → Not accessible outside the class (not inherited).
- **default (package-private)** → Accessible only within the same package.
- **protected** → Accessible within the same package **and** by subclasses (even in different packages).
- **public** → Accessible from anywhere.

### 15. Can we achieve multiple inheritance in Java? If yes, how?

- ✕ Not with classes → Java avoids **diamond problem**.
- ✅ Achieved using **interfaces**.

```
interface A { void methodA(); }
interface B { void methodB(); }

class C implements A, B {
    public void methodA() { System.out.println("A"); }
    public void methodB() { System.out.println("B"); }
}
```

### 16. Explain the concept of covariant return types in overriding.

- When overriding, the return type of the child method can be a **subclass** of the parent's method return type.

```
class Animal {}
class Dog extends Animal {}

class Parent {
    Animal getAnimal() { return new Animal(); }
}

class Child extends Parent {
    @Override
    Dog getAnimal() { return new Dog(); } // covariant return
}
```

### 17. What is the difference between composition and inheritance? Which one is preferred?

- **Inheritance** → "is-a" relationship
  - Example: `Dog is an Animal`
  - Less flexible, creates tight coupling.
- **Composition** → "has-a" relationship
  - Example: `Car has an Engine`
  - More flexible, preferred over inheritance in most cases (looser coupling & better code reuse).

✅ **Preferred: Composition**

### 18. How does `instanceof` operator work with inheritance?

- `instanceof` checks at **runtime** if an object is an instance of a class or its subclass.

```
Animal a = new Dog();
System.out.println(a instanceof Animal); // true
System.out.println(a instanceof Dog);    // true
```

### 19. Explain the Template Method Pattern with an example in OOPS.

- **Definition**: Defines the **skeleton of an algorithm** in a parent class, but lets child classes provide implementations for certain steps.

```
abstract class Payment {
    public final void makePayment() {
        validate();
```

```
        process();
        receipt();
    }

    abstract void process();

    void validate() { System.out.println("Validating payment"); }
    void receipt() { System.out.println("Printing receipt"); }
}

class CardPayment extends Payment {
    void process() { System.out.println("Processing card payment"); }
}
```

- Here:
  - `makePayment()` = **Template Method** (defines algorithm steps).
  - Subclass (`CardPayment`) overrides `process()`.

## 20. What are default methods in interfaces? Why were they introduced?

- **Introduced in Java 8**.
- Allow interfaces to have **implemented methods** using `default`.

**Purpose**:

1. Backward compatibility → Add new methods to interfaces without breaking old implementations.
2. Enable **multiple inheritance of behavior** along with classes & other interfaces.

```
interface Vehicle {
    default void start() {
        System.out.println("Vehicle starting...");
    }
}
```

✅ Now this is fully cleaned up and ready in **Markdown format**.

Do you also want me to create a **summary table (Q11–Q20 in one glance)** at the end, for quick revision before exams?